객체지향 프로그래밍 고급

11강 : 연산자오버로딩

Chapter 11

연산자 오버로딩의 이해

단항연산자의 오버로딩

<<,>>연산자 오버로딩

연산자 오버로딩의 이해

```
class Point {
    int xpos;
    int ypos;
public:
    Point(int x, int y){ ..}
};
int main()
{
    Point pos1(10, 20), pos2(20, 40);
    . . .
    return 0;
}
```



```
Point pos1(10, 20);
Point pos2(20, 40);
Point pos3 = pos1 + pos2;
```

위 연산의 결과, pos3가 (30, 60)의 값을 갖는가?

연산자 오버로딩의 이해

- 두 객체의 값을 더하려면 Point 클래스에 Sum() 함수가 필요하다.

```
Point Point::Sum(Point right)
{
    Point pos;
    pos.xpos = xpos + right.xpos;
    pos.ypos = ypos + right.ypos;
}
```



```
Point pos1(10, 20);
Point pos2(20, 40);
Point pos3 = pos1.Sum(pos2);
```

위 코드를
Point pos3 = pos1 + pos2;
로 표현해 보자!!

연산자 오버로딩(operator overloading)

연산자를 클래스에서 객체 연산에 사용할 수 있도록 그 의미를 다시 부여하는 것을 “연산자 오버로딩“ 도는 “연산자 중복(재정의)”라고 함

- 연산자 오버로딩은 기본적으로 함수의 오버로딩과 같이 연산자도 하나의 함수라는 개념을 사용하여 중복 정의
- 오버로딩되는 연산자 함수는 클래스의 멤버함수나 프렌드 함수로 정의
- 함수 이름 대신에 operator 키워드를 사용하고 다음에 연산자를 기술

연산자 오버로딩

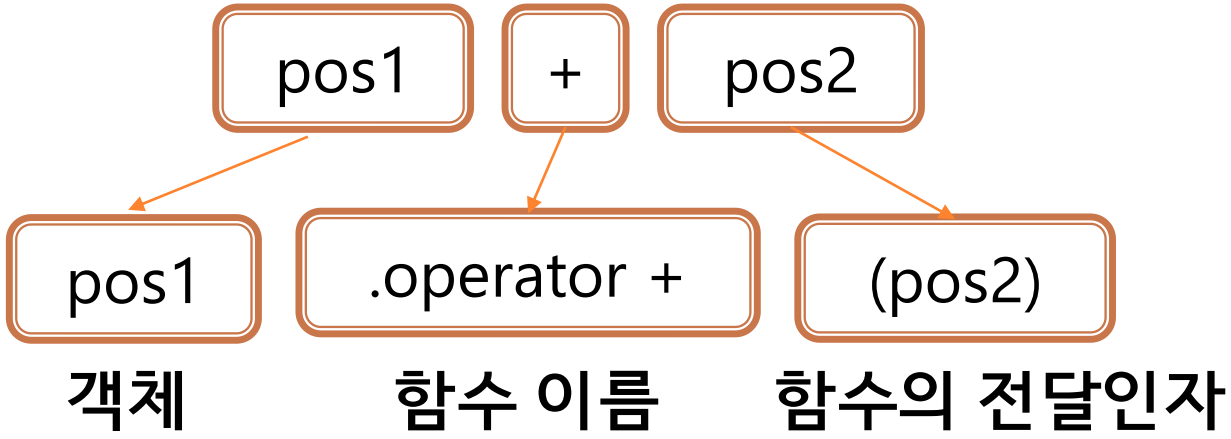
```
class Point {
public:
    Point operator+(Point obj);
    . . . .
};
Point Point::operator+(const Point obj)
{
    . . . .
}

int main() {
    Point pt1, pt2, pt3;
    pt3 = pt1 + pt2;
    . . . .
}
```

- operator+는 함수로 간주
- pt3=pt1+pt2; 는 pt3=pt1.operator+(pt2); 와 같음

```
Point operator+(const Point &ref) {
    Point tmp;
    tmp.xpos = xpos + ref.xpos;
    tmp.ypos = ypos + ref.ypos;
    return tmp;
}
```

연산자 오버로딩



연산자 오버로딩의 예

Point pos3 = pos1 + pos2;

```
class Point {
private:
	int xpos, ypos;
public:
	Point() {
		xpos = ypos = 0;
	}
	Point(int x, int y) : xpos(x), ypos(y) {  }
	void ShowPosition() const {
		cout << xpos << " , " << ypos << endl;
	}
	Point operator+(const Point &ref) {
		Point tmp;
		tmp.xpos = xpos + ref.xpos;
		tmp.ypos = ypos + ref.ypos;
		return tmp;
	}
};
```

```
int main()
{
	Point pos1(3, 4);
	Point pos2(10, 20);
	Point pos3 = pos1.operator+(pos2);
	pos1.ShowPosition();
	pos2.ShowPosition();
	pos3.ShowPosition();
	return 0;
}
```

피연산자의 값을 변경하는 것이 아니므로 **const** 사용

주소참조의 목적이 아니라 메모리 낭비를 막기 위해 참조자 사용

[예제1] –연산자 오버로딩

Point 클래스에 대해 – 연산자를 오버로딩하시오.

[예제2]+= 연산자 오버로딩

Point 클래스에 대해 += 연산자를 오버로딩하시오.

```
pos1 += pos2;
```

```
#include <iostream>
using namespace std;

class Point {
private:
	int xpos, ypos;
public:
	Point() {
		xpos = ypos = 0;
	}
	Point(int x, int y) : xpos(x), ypos(y) {  }
	void ShowPosition() const {
		cout << xpos << " , " << ypos << endl;
	}
	Point& operator+=(const Point &ref) {
		xpos += ref.xpos;
		ypos += ref.ypos;
		return *this;
		}
};
```

```
int main()
{
	Point pos1(3, 4);
	Point pos2(10, 20);
	pos1 += pos2;
	pos1.ShowPosition();
	return 0;
}
```

[예제3] == 연산자와 != 연산자

연산 pos1 == pos2의 결과로 모든 멤버(xpos, ypos)의 값이 같다면 true, 그렇지 않다면 false를 반환

```
#include <iostream>
using namespace std;

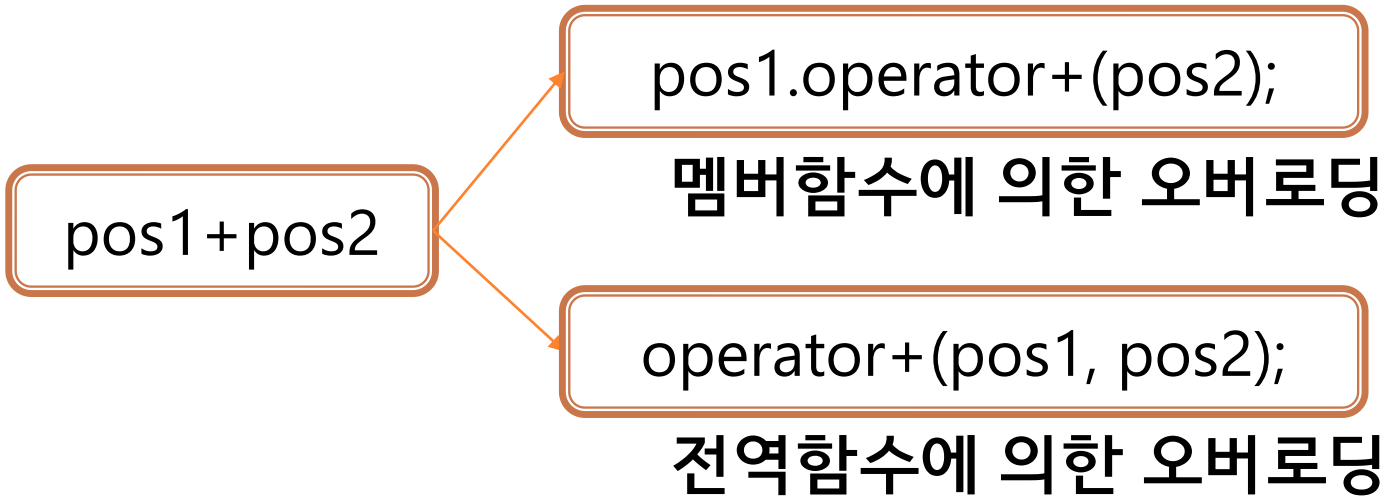
class Point {
private:
	int xpos, ypos;
public:
	Point() {
		xpos = ypos = 0;
	}
	Point(int x, int y) : xpos(x), ypos(y) {  }
	void ShowPosition() const {
		cout << xpos << " , " << ypos << endl;
	}
	bool operator==(Point &ref) {
		if (xpos == ref.xpos && ypos == ref.ypos)
			return true;
		else
			return false;
	}
};
```

```
int main()
{
	Point pos1(3, 4);
	Point pos2(10, 20);
	if (pos1 == pos2)
		cout << "pos1과 pos2가 같다\n";
	else
		cout << "pose1과 pos2가 같지 않다.\n";
	return 0;
}
```

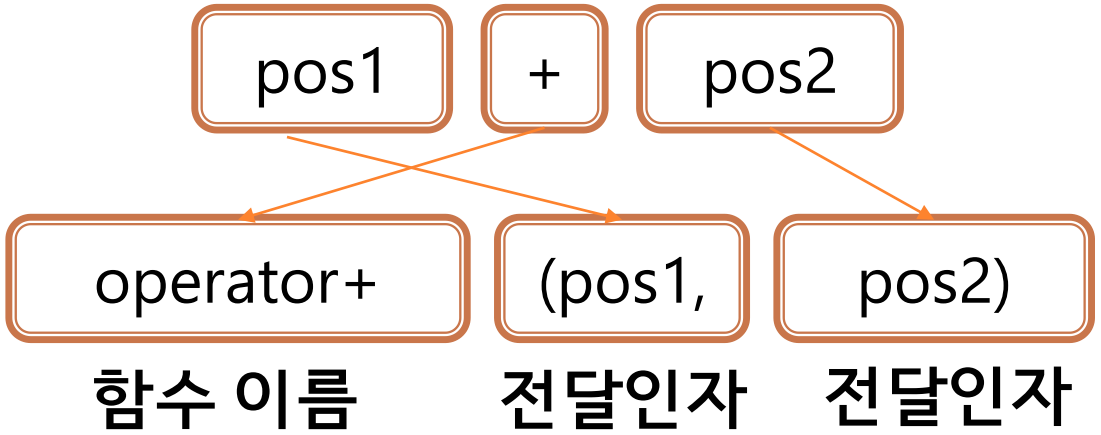
연산자를 오버로딩하는 방법

①멤버함수에 의한 연산자 오버로딩

②전역함수에 의한 연산자 오버로딩



전역함수에 의한 오버로딩



전역함수에 의한 연산자 오버로딩 방법

```
class Point {
    . . .

   friend Point operator+(const Point &pos1, const Point &pos2);
};

Point operator+(const Point &pos1, const Point &pos2)
{
     Point pos(pos1.xpos + pos2.xpos, pos1.ypos + pos2.ypos);
     return pos;
}
```

전역함수에 의한 연산자 오버로딩의 예

```
class Point {
private:
	int xpos, ypos;
public:
	Point() {
		xpos = ypos = 0;
	}
	Point(int x, int y) : xpos(x), ypos(y) {  }
	void ShowPosition() const {
		cout << xpos << " , " << ypos << endl;
	}
	friend Point operator+(const Point &pos1, const
	Point &pos2);
};
```

```
Point operator+(const Point &pos1, const Point &pos2)
{
	Point pos(pos1.xpos + pos2.xpos, pos1.ypos +
	pos2.ypos);
	return pos;
}

int main()
{
	Point pos1(3, 4);
	Point pos2(10, 20);
	Point pos3 = pos1 + pos2;
	pos1.ShowPosition();
	pos2.ShowPosition();
	pos3.ShowPosition();
	return 0;
}
```

[예제4]전역함수 기반 오버로딩

- Point 클래스에 대해서 전역함수 기반으로 – , ==, != 연산자를 오버로딩하시오.

```
#include <iostream>
using namespace std;

class Point {
private:
	int xpos, ypos;
public:
	Point() {
		xpos = ypos = 0;
	}
	Point(int x, int y) : xpos(x), ypos(y) {  }
	void ShowPosition() const {
		cout << xpos << " , " << ypos << endl;
	}
	friend Point operator-(const Point &pos1, const Point
	&pos2);
	friend bool operator==(const Point &pos1, const Point
	&pos2);
	friend bool operator!=(const Point &pos1, const Point
	&pos2);
};
Point operator-(const Point &pos1, const Point &pos2)
{
		Point pos(pos1.xpos - pos2.xpos, pos1.ypos -
pos2.ypos);
		return pos;
}
bool operator== (const Point &pos1, const Point &pos2)
{
		if (pos1.xpos == pos2.xpos &&
			pos1.ypos == pos2.ypos) return true;
		else
			return false;
}

int main()
{
	Point pos1(20, 30);
	Point pos2(10, 20);
	Point pos3(10, 20);

	(pos1 - pos2).ShowPosition();
	if (pos2 == pos3)
		cout << "equal" << endl;
	else
		cout << "not equal" << endl;
	return 0;
}
```

```
class point {
. . .
    friend Point& operator+=(Point&, Point&);
};

Point& operator+=(Point& pos1, Point& pos2)
{
    pos1.xpos+=pos2.xpos;
    pos1.ypos+=pos2.ypos;
    return pos1;
}
```

```
int main()
{
    ....
    (pos1+=pos2).ShowPosition();
    ....
}
```

멤버함수 기반으로만 오버로딩이 가능한 연산자

=	대입연산자
()	함수호출연산자
[ ]	배열접근연산자
->	포인터연산자

연산자 오버로딩 시 주의사항

- 본래의 의도를 벗어난 형태의 연산자 오버로딩은 좋지 않다.
- 연산자의 우선순위와 결합성은 바뀌지 않는다.
- 매개변수의 디폴트 값 설정은 불가능하다.
- 연산자의 순수기능은 유지된다.(+연산자를 *로 쓰지 않음)

[예제5]

main프로그램이 실행될 수 있도록 두 수를 더하는 연산자 +를 재정의하시오.

```
class Number {
private:
    int num;
public:
    Number();
    Number(int data) : num(data){ }
    Number operator+(Number n);
    void Show() { cout << num << endl;}
```

```
int main()
{
    Number a(5);
    Number b(3);
    Number c;
    c = a + b;
    c.Show();
    return 0;
```

[예제6]+연산자 오버로드

아래와 같은 멤버변수를 갖는 Cube 클래스와 +연산자가 있다. 아래와 같은 결과가 나오도록 코드를 완성하시오.

```
class Cube {
private:
    double x;
    double y;
    double z;
...
};
```

```
int main()
{
    Cube c = Cube(12.2, 89.4, 5.0);
    Cube d = Cube(67.3, 53.2, 1.3);
    Cube e = c + d;
    c.list("Welcome");
    d.list("Vision");
    e.list("Passion");
    return 0;
}
```

```
Welcome 12.2 , 89.4 , 5
Vision 67.3 , 53.2 , 1.3
Passion 67.3 , 89.4 , 6.3
계속하려면 아무 키나 누르십시오 . . .
```

e = c + d;
⇒ c클래스의 x, y와 d클래스의 x, y 중 큰값을 반환
⇒ c클래스의 z와 d클래스의 z의 합을 반환

[예제7]

서로 다른 자료형의 데이터 연산을 위한 연산자 오버로딩을 구현하시오.

```
class Point
{
private:
    int xpos, ypos;
. . .
}
```

```
int main()
{
    Point pos(1, 2);
    Point cpy;

    cpy = pos * 3;
    cpy.ShowPosition();

    cpy = pos * 3 * 2;
    cpy.ShowPosition();
    return 0;
}
```

[예제8]

다음 연산이 가능하도록 연산자*를 오버로딩하시오.

```
int main()
{
    Point pos(20, 30);
    Point cpy;

    cpy = 3 * pos;
    cpy.ShowPosition();
    cpy = 2 * pos * 3;
    cpy.ShowPosition();

return 0;
}
```

```
class Point {
    int xpos, ypos;
public:
    Point() { xpos = ypos = 0; }
    Point(int x, int y) :xpos(x), ypos(y) {}
    Point operator*(int n)
    {
        return Point(xpos*n, ypos*n);
    }
    void ShowPosition()
    {
        cout << xpos << "," << ypos << endl;
    }
    friend Point operator*(int times, Point& ref);
};
Point operator*(int times, Point& ref)
{
    return ref * times;
}
```

cpy=3*pos;
cpy=operator*(3,pos);
3*pos를 구현하기 위해서는 전역함수의 형태로 곱셈연산자를 오버로딩해야함

```
Point operator*(int times, Point &ref)
{ Point pos(ref.xpos*times,ref.ypos*times);
  return pos;
}
```

⇒ 오버로딩

```
Point opertator*(int times, Point &ref)
{
  return ref*times;
}
```

Chapter 11

연산자 오버로딩의 이해

단항연산자의 오버로딩

<<,>>연산자 오버로딩

단항 연산자의 오버로딩

- **1 증가 연산자 ++**

  ++pos; → pos.operator++(); , 전역함수 : operator++(pos);

- **1 감소 연산자 --**

  --pos; → pos.operator - -(); , 전역함수 : operator- -(pos);

단항 연산자의 오버로딩의 예

```
class Point {
private:
	int xpos, ypos;
public:
	Point() {
		xpos = ypos = 0;
	}
	Point(int x, int y) : xpos(x), ypos(y) {  }
	void ShowPosition() const {
		cout << xpos << " , " << ypos << endl;
	}
	Point& operator++() {
		xpos += 1;
		ypos += 1;
		return *this;
	}
	friend Point& operator--(Point& ref);
};
```

```
Point& operator--(Point &ref) {
		ref.xpos -= 1;
		ref.ypos -= 1;
		return ref;
}
```

```
int main()
{
	Point pos1(3, 4);
	Point pos2(10, 20);
	++pos1;
	pos1.ShowPosition();
	(--pos2).ShowPosition();

	return 0;
}
```

단항 연산자의 오버로딩

객체자신을 참조할 수 있는 참조값이 반환

```
Point& operator++()
{
    xpos += 1;
    ypos += 1;
    return *this;
}
```

```
Point& operator--(Point &ref) {
        ref.xpos -= 1;
        ref.ypos -= 1;
        return ref;
}
```

[예제9] 단항연산자 -

- 연산자는 피연산자의 부호를 반전시킨 결과를 반환한다.

```
int num2 = -num1;
```

num2에는 num1과 절대값은 같지만 부호가 다른 값이 저장된다.

Point 클래스를 대상으로 – 연산자를 오버로딩하라.

```
Point pos2 = -pos1;
```

(이 때, pos1의 값은 변하지 않는다.)

```cpp
#include <iostream>
using namespace std;

class Point {
private:
    int xpos, ypos;
public:
    Point() {
        xpos = ypos = 0;
    }
    Point(int x, int y) : xpos(x), ypos(y) {  }
    void ShowPosition() const {
        cout << xpos << " , " << ypos << endl;
    }
    Point operator-() {
        Point pos(-xpos, -ypos);
        return pos;
    }
};
```

```cpp
int main()
{
    Point pos1(3, 4);
    Point pos2 = -pos1;
    pos1.ShowPosition();
    pos2.ShowPosition();
    return 0;
}
```

전위증가와 후위증가

++pos → pos.operator++();
pos++ → pos.operator++(int);

--pos → pos.operator--();
Pos-- → pos.operator--(int);

```
#include <iostream>
using namespace std;

class Point {
private:
    int xpos, ypos;
public:
    Point() {
        xpos = ypos = 0;
    }
    Point(int x, int y) : xpos(x), ypos(y) {  }
    void ShowPosition() const {
        cout << xpos << " , " << ypos << endl;
    }
    Point& operator++() {
        xpos += 1;
        ypos += 1;
        return *this;
    }
    const Point operator++(int) {
        const Point obj(xpos, ypos);
        xpos += 1;
        ypos += 1;
        return obj;
    }
};

int main()
{
    Point pos1(3, 4);
    Point cpy = pos1++;
    pos1.ShowPosition();
    cpy.ShowPosition();
    cpy = ++pos1;
    pos1.ShowPosition();
    cpy.ShowPosition();
    return 0;
}
```

반환형에서의 const 선언과 const 객체

앞의 예제에서 operator++ 함수의 반환형으로 const를 선언한 이유:

```
①const Point operator++(int) {
    ②const Point obj(xpos, ypos);// 함수내에서 obj의 변경을
                                  //막기 위해
    xpos += 1;
    ypos += 1;
    return obj;
}
```

함수의 반환으로 인해서 생성되는 임시객체를 const 객체로 생성함(값의 변경을 허용하지 않음)

```
int main()
{
    point pos(3, 5);
    (pos++)++;      컴파일에러
    ....
}
```

[예제10] 전역함수를 이용한 전위감소, 후위감소 연산자 오버로딩

```
#include <iostream>
using namespace std;

class Point {
private:
    int xpos, ypos;
public:
    Point() {
        xpos = ypos = 0;
    }
    Point(int x, int y) : xpos(x), ypos(y) {  }
    void ShowPosition() const {
        cout << xpos << " , " << ypos << endl;
    }
    friend Point& operator--(Point &ref);
    friend const Point operator--(Point &ref, int);
};
```

```
Point& operator--(Point &ref)
{
    ref.xpos--=1;
    ref.ypos--=1;
    return ref;
}
const Point operator--(Point &ref, int) {
        const Point obj(ref);
        ref.xpos -= 1;
        ref.ypos -= 1;
        return obj;
}

int main()
{
    Point pos1(3, 4);
    Point cpy = pos1--;
    pos1.ShowPosition();
    cpy.ShowPosition();
    cpy = --pos1;
    pos1.ShowPosition();
    cpy.ShowPosition();
    return 0;
}
```

[예제11]

++ 연산자를 오버로딩하여 다음과 같은 결과를 출력하시오.

```
class My
{
    int val;
public:
    My(int data) :val(data) {}
    void list() { cout << val << endl; }
    . . .
};
int main()
{
    My a = 5;
    My x = a++;
    a.list();
    x.list();
    My y = ++a;
    a.list();
    y.list();
    return 0;
}
```

```
6
5
7
7
```

```
class My
{
    int val;
public:
    My(int data) :val(data) {}
    void list() { cout << val << endl; }
    const My& operator++() {
        ++val;
        return *this;
    }
    const My operator++(int) {
        My org(*this); // My org = *this;
        ++(*this);     // ++val;
        return org;
    }
};
```

[예제12]

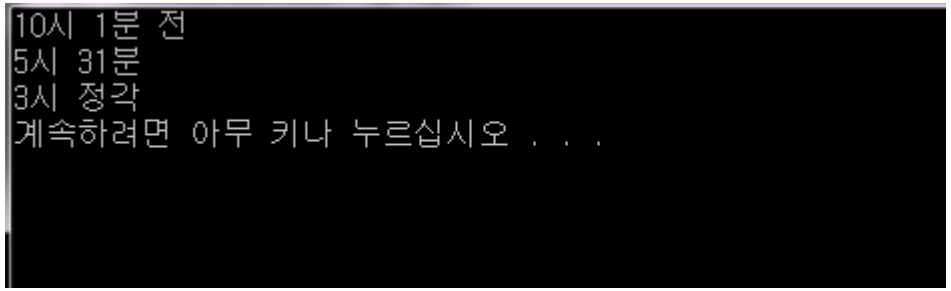
Time은 시, 분을 의미하는 클래스이다.
증가연산자는 현재 시간에서 1분을 더하며, 전위증가 연산자는 “x시 x분 전“으로 표시하고 후위증가 연산자는 “x시 x분 " 으로 표시한다. ++ 연산자를 오버로딩하여 다음과 같은 결과를 출력하시오.

```
int main()
{
	Time t1(9, 58), t2(5, 30), t3(2,59);
	++t1;
	t1.display();

	t2++;
	t2.display();

	t3++;
	t3.display();

	return 0;
}
```



```
#include <iostream>
using namespace std;

class Time
{
	int hours;
	int minutes;
public:
	Time() : hours(0), minutes(0) {}
	Time(int h, int m) :hours(h), minutes(m) {}
	void display()
	{

		if (minutes == 0)
			cout << hours << "시 " << "정각" << endl;
		else if (minutes < 0)
			cout << hours << "시 " << -minutes << “
			분 전"	<< endl;

		else
			cout << hours << "시 " << minutes << “
			분"		<< endl;

	}
	Time operator++()
	{
		++minutes;
		if (minutes < 60)
		{
			++hours;
			minutes -= 60;
		}
		return Time(hours, minutes);
	}
	Time operator++(int)
	{
		Time T(hours, minutes);
		++minutes;
		if (minutes >= 60)
		{
			++hours;
			minutes -= 60;
		}
		return T;
	}
};
```